

Problem

Submissions

Leaderboard

Discussions

Editorial

For two strings A and B, we define the similarity of the strings to be the length of the longest prefix common to both strings. For example, the similarity of strings "abc" and "abd" is 2, while the similarity of strings "aab" and "baab" is 3. Calculate the sum of similarities of a string S with each of its suffixes.

Input Format

The first line contains the number of test cases t. Each of the next t lines contains a string to process, s.

Constraints

- 1 ≤ t ≤ 10
- 1 ≤ |s| ≤ 100000
- s is composed of characters in the range ascii[a-z]

Output Format

Output t lines, each containing the answer for the corresponding test case.

Sample Input

```
2
ababaa
aa
```

Sample Output

```
11
3
```

Explanation

For the first case, the suffixes of the string are "ababaa", "babaa", "abaaa", "baaa", "aaaa" and "a". The similarities of these strings with the string "ababaa" are 6,0,3,0,1, & 1 respectively. Thus, the answer is 6 + 0 + 3 + 0 + 1 + 1 = 11.

For the second case, the answer is 2 + 1 = 3.

```
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48

public static int stringSimilarity(String s) {
    int n = s.length();
    int[] z = new int[n];

    int left = 0;
    int right = 0;

    for (int i = 1; i < n; i++) {
        if (i <= right) {
            z[i] = Math.min(right - i + 1, z[i - left]);
        }

        while (i + z[i] < n &&
                s.charAt(z[i]) == s.charAt(i + z[i])) {
            z[i]++;
        }

        if (i + z[i] - 1 > right) {
            left = i;
            right = i + z[i] - 1;
        }
    }

    int result = n; // similarity with the string itself

    for (int i = 1; i < n; i++) {
        result += z[i];
    }
}
```

Line: 26 Col: 1

Upload Code as File ☐ Test against custom input

Run Code

Submit Code

Congratulations!
You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Problem

Submissions

Leaderboard

Discussions

Editorial

Given two strings, determine if they share a common substring. A substring may be as small as one character.

Example

s1 = 'aud'
s2 = 'art'

These share the common substring a.

s1 = 'be'
s2 = 'cat'

These do not share a substring.

Function Description

Complete the function twoStrings in the editor below.

twoStrings has the following parameter(s):

- string s1: a string
- string s2: another string

Returns

- string: either YES or NO

Input Format

The first line contains a single integer *p*, the number of test cases.

The following *p* pairs of lines are as follows:

- The first line contains string *s1*.
- The second line contains string *s2*.

Constraints

- s1* and *s2* consist of characters in the range `ascii[a-z]`.
- $1 \leq p \leq 10$
- $1 \leq |s1|, |s2| \leq 10^5$

Output Format

For each pair of strings, return YES or NO.

Sample Input

```
10
11
12 /*
13  * Complete the 'twoStrings' function below.
14  * The function is expected to return a STRING.
15  * The function accepts following parameters:
16  * 1. STRING s1
17  * 2. STRING s2
18  */
19
20 public static String twoStrings(String s1, String s2) {
21
22     boolean[] seen = new boolean[26];
23
24     // Mark characters present in s1
25     for (int i = 0; i < s1.length(); i++) {
26         seen[s1.charAt(i) - 'a'] = true;
27     }
28
29     // Check whether any character of s2 is also in s1
30     for (int i = 0; i < s2.length(); i++) {
31         if (seen[s2.charAt(i) - 'a']) {
32             return "YES";
33         }
34     }
35
36     return "NO";
37 }
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Line: 64 Col: 1

☐ Test against custom input

Congratulations!
You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Problem List

796. Rotate String

EasyTopicsCompanies

Given two strings `s` and `goal`, return `true` if and only if `s` can become `goal` after some number of **shifts** on `s`.

A **shift** on `s` consists of moving the leftmost character of `s` to the rightmost position.

For example, if `s` = "abcde", then it will be "bcdea" after one shift.

Example 1:
Input: `s` = "abcde", `goal` = "cdeab"
Output: `true`

Example 2:
Input: `s` = "abcde", `goal` = "abced"
Output: `false`

Constraints:

- `1 <= s.length, goal.length <= 100`
- `s` and `goal` consist of lowercase English letters.

UNLOCK EXPERIENCES
Journey to a self-driving network
HPE

Seen this question in a real interview before? 1/6

YesNo

Accepted 1,013,002 / 1.5M | Acceptance Rate 66.6%

5.1K303

Solved

Code

JavaAuto

```
1
2 class Solution {
3     public boolean rotateString(String s, String goal) {
4         // Different lengths can never be rotations
5         if (s.length() != goal.length()) {
6             return false;
7         }
8         // Every rotation of s will appear in s + s
9         return (s + s).contains(goal);
10    }
11 }
12
13
```

Saved

TestcaseTest Result

Accepted Runtime: 0 ms

Case 1Case 2

Input

Problem List

DescriptionEditorialSolutionsSubmissions

459. Repeated Substring Pattern

EasyTopicsCompanies

Given a string `s`, check if it can be constructed by taking a substring of it and appending multiple copies of the substring together.

Example 1:

Input: `s = "abab"`
Output: `true`
Explanation: It is the substring "ab" twice.

Example 2:

Input: `s = "aba"`
Output: `false`

Example 3:

Input: `s = "abcabcabcabc"`
Output: `true`
Explanation: It is the substring "abc" four times or the substring "abcabc" twice.

Constraints:

- `1 <= s.length <= 104`
- `s` consists of lowercase English letters.

DISCOVER

DURI

Seen this question in a real interview before? 1/5

6.9K159

Solved

Code

JavaAuto

```
1 class Solution {
2     public boolean repeatedSubstringPattern(String s) {
3
4         int n = s.length();
5
6         for (int len = 1; len <= n / 2; len++) {
7
8             if (n % len != 0)
9                 continue;
10
11             String sub = s.substring(0, len);
12             StringBuilder result = new StringBuilder();
13
14             for (int i = 0; i < n / len; i++) {
15                 result.append(sub);
16             }
17
18             if (result.toString().equals(s))
19                 return true;
20
21         }
22
23         return false;
24     }
```

Testcase

Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Case 3

Input

A space explorer's ship crashed on Mars! They send a series of SOS messages to Earth for help.



Letters in some of the SOS messages are altered by cosmic radiation during transmission. Given the signal received by Earth as a string, *s*, determine how many letters of the SOS message have been changed by radiation.

Example

s = "SOSTOT"

The original message was SOSOSOS. Two of the message's characters were changed in transit.

Function Description

Complete the marsExploration function in the editor below.

```
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
/*
 * Complete the 'marsExploration' function below.
 * The function is expected to return an INTEGER.
 * The function accepts STRING s as parameter.
 */

public static int marsExploration(String s) {

    int count = 0;
    String pattern = "SOS";

    for (int i = 0; i < s.length(); i++) {
        if (s.charAt(i) != pattern.charAt(i % 3)) {
            count++;
        }
    }

    return count;
}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));
    }
}
```

Line: 58 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Output (stdout)

Download

438. Find All Anagrams in a String

Medium Topics Companies

Given two strings *s* and *p*, return an array of all the start indices of *p*'s anagrams in *s*. You may return the answer in **any order**.

Example 1:

Input: *s* = "cbaebabacd", *p* = "abc"
Output: [0,6]
Explanation:
The substring with start index = 0 is "cba", which is an anagram of "abc".
The substring with start index = 6 is "bac", which is an anagram of "abc".

Example 2:

Input: *s* = "abab", *p* = "ab"
Output: [0,1,2]
Explanation:
The substring with start index = 0 is "ab", which is an anagram of "ab".
The substring with start index = 1 is "ba", which is an anagram of "ab".
The substring with start index = 2 is "ab", which is an anagram of "ab".

Constraints:

- $1 \leq s.length, p.length \leq 3 \times 10^4$
- s* and *p* consist of lowercase English letters.



Seen this question in a real interview before? 1/6

13.4K 187 1 1 1

Solved

Code

Java Auto

```
1
2 import java.util.*;
3
4 class Solution {
5     public List<Integer> findAnagrams(String s, String p) {
6         List<Integer> result = new ArrayList<>();
7
8         if (s.length() < p.length()) {
9             return result;
10        }
11
12        int[] pCount = new int[26];
13        int[] windowCount = new int[26];
14
15        // Count characters in p
16        for (char c : p.toCharArray()) {
17            pCount[c - 'a']++;
18        }
19
20        int windowSize = p.length();
21
22        // Create the first window
23        for (int i = 0; i < windowSize; i++) {
24            windowCount[s.charAt(i) - 'a']++;
25        }
26
27        // Check the first window
28        if (Arrays.equals(pCount, windowCount)) {
29            result.add(0);
30        }
31
32        // Slide the window
33        for (int i = windowSize; i < s.length(); i++) {
34            // Add new character
35            windowCount[s.charAt(i) - 'a']++;
36            // Remove old character
37            windowCount[s.charAt(i - windowSize) - 'a']--;
38        }
39
40        for (int i = windowSize; i < s.length(); i++) {
41            if (Arrays.equals(pCount, windowCount)) {
42                result.add(i);
43            }
44        }
45    }
46}
```

Saved

Ln 1, Col 1

Testcase Test Result

Accepted

Runtime: 0 ms.

Case 1

Case 2

Problem

Submissions

Leaderboard

Discussions

Editorial

Given a string of lowercase letters in the range `ascii[a-z]`, determine the index of a character that can be removed to make the string a `palindrome`. There may be more than one solution, but any will do. If the word is already a `palindrome` or there is no solution, return `-1`. Otherwise, return the index of a character to remove.

Example

`s = "bcbe"`

Either remove 'b' at index 0 or 'c' at index 3.

Function Description

Complete the `palindromeIndex` function in the editor below.

`palindromeIndex` has the following parameter(s):

- `string s`: a string to analyze

Returns

- `int`: the index of the character to remove or `-1`

Input Format

The first line contains an integer `q`, the number of queries.

Each of the next `q` lines contains a query string `s`.

Constraints

- $1 \leq q \leq 20$
- $1 \leq \text{length of } s \leq 10^5 + 5$
- All characters are in the range `ascii[a-z]`.

Sample Input

STDIN	Function
3	q = 3
aaab	s = 'aaab' (first query)
baa	s = 'baa' (second query)
aaa	s = 'aaa' (third query)

Sample Output

```
15  * The function accepts STRING s as parameter.
16  */
17
18  public static int palindromeIndex(String s) {
19
20      int left = 0;
21      int right = s.length() - 1;
22      while (left < right) {
23
24          if (s.charAt(left) != s.charAt(right)) {
25
26              // Try removing the left character
27              if (isPalindrome(s, left + 1, right)) {
28                  return left;
29              }
30
31              // Try removing the right character
32              if (isPalindrome(s, left, right - 1)) {
33                  return right;
34              }
35
36              // No solution
37              return -1;
38          }
39
40          left++;
41          right--;
42      }
43  }
```

Line: 84 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Compiler Message

28. Find the Index of the First Occurrence in a String

Solved

Easy Topics Companies

Given two strings `needle` and `haystack`, return the index of the first occurrence of `needle` in `haystack`, or `-1` if `needle` is not part of `haystack`.

Example 1:

Input: `haystack = "sadbutsad", needle = "sad"`
Output: `0`
Explanation: "sad" occurs at index 0 and 6.
The first occurrence is at index 0, so we return 0.

Example 2:

Input: `haystack = "leetcode", needle = "leeto"`
Output: `-1`
Explanation: "leeto" did not occur in "leetcode", so we return -1.

Constraints:

- $1 \leq \text{haystack.length}, \text{needle.length} \leq 10^4$
- `haystack` and `needle` consist of only lowercase English characters.



Seen this question in a real interview before? 1/6

Yes No

Accepted 4,329,982/9.2M Acceptance Rate 47.3%

7.8K 581 ☆

Code

Java Auto

```
1 class Solution {
2     public int strStr(String haystack, String needle) {
3         int n = haystack.length();
4         int m = needle.length();
5
6         for (int i = 0; i <= n - m; i++) {
7             int j = 0;
8
9             while (j < m && haystack.charAt(i + j) == needle.charAt(j)) {
10                 j++;
11             }
12
13             if (j == m) {
14                 return i;
15             }
16         }
17
18         return -1;
19     }
20 }
21 }
```

Saved

Ln 1, Col 1

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Description Editorial Solutions Submissions

5. Longest Palindromic Substring

Medium Topics Companies Hint

Given a string s , return the longest *palindromic substring* in s .

Example 1:

Input: $s = \text{"babad"}$
Output: "bab"
Explanation: "aba" is also a valid answer.

Example 2:

Input: $s = \text{"cbdd"}$
Output: "bb"

Constraints:

- $1 \leq s.length \leq 1000$
- s consist of only digits and English letters.

 Hero
Scrap Old Bike For Value
Learn More

Seen this question in a real interview before? 1/5

Yes No

Accepted 5,054,490 / 13.1M Acceptance Rate 38.6%

Topics

33.1K 678 ☆ ↗ ↻

</> Code

Java Auto

```
1 class Solution {
2     public String longestPalindrome(String s) {
3         if (s.length() < 2) return s;
4
5         int start = 0, end = 0;
6
7         for (int i = 0; i < s.length(); i++) {
8             int len1 = expand(s, i, i); // Odd length
9             int len2 = expand(s, i, i + 1); // Even length
10
11             int len = Math.max(len1, len2);
12
13             if (len > end - start + 1) {
14                 start = i - (len - 1) / 2;
15                 end = i + len / 2;
16             }
17         }
18         return s.substring(start, end + 1);
19     }
20
21     private int expand(String s, int left, int right) {
22         while (left >= 0 && right < s.length() &&
23               s.charAt(left) == s.charAt(right)) {
24             left--;
25             right++;
26         }
27         return right - left - 1;
28     }
29 }
30
31 }
```

Saved

Ln 31, Col 2

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Problem

Submissions

Leaderboard

Discussions

A palindrome is a string that reads the same from left to right as it does from right to left.
Given a string, S , of N lowercase English letters, we define a k -length rotation as cutting the first k characters from the beginning of S and appending them to the end of S . For each S , there are N possible k -length rotations (where $0 \leq k < N$). See the Explanation section for examples.

Given N and S , find all N k -length rotations of S ; for each rotated string, S_k , print the maximum possible length of any palindromic substring of S_k on a new line.

Input Format

The first line contains an integer, N (the length of S).
The second line contains a single string, S .

Constraints

- $1 \leq N \leq 5 \times 10^5$
- $0 \leq k < N$
- S is comprised of lowercase English letters.

Output Format

There should be N lines of output, where each line k contains an integer denoting the maximum length of any palindromic substring of rotation S_k .

Sample Input 0

```
13
aaaaabbbbbaaa
```

Sample Output 0

```
12
12
10
8
8
9
11
13
```

```
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
List<Integer> result = new ArrayList<>();
int n = s.length();

// Create circular string
String str = s + s;

for (int start = 0; start < n; start++) {
    int max = 1;

    // Check every possible palindrome center
    for (int center = start; center < start + n; center++) {

        // Odd length palindrome
        int left = center;
        int right = center;

        while (left >= start && right < start + n &&
                str.charAt(left) == str.charAt(right)) {
            max = Math.max(max, right - left + 1);
            left--;
            right++;
        }

        // Even length palindrome
        left = center;
        right = center + 1;

        while (left >= start && right < start + n &&
                str.charAt(left) == str.charAt(right)) {
            max = Math.max(max, right - left + 1);
            left--;
            right++;
        }
    }
}
```

Line: 29 Col: 5

Upload Code as File ☐ Test against custom input

Run Code Submit Code

Congratulations!
You have passed the sample test cases. Click the submit button to run your code against all the test cases.